

**Universidad Simón Bolívar**  
Decanato de Estudios Profesionales  
Departamento de Computación y Tecnología de la Información  
CI2692 Laboratorio de Algoritmos y Estructuras de Datos II  
Trimestre Abril-julio 2026

Profesores: Leonardo López - Kevin Oporto

Mayo 2026

## Laboratorio 2: Mergesort y Visualización

### 1. Introducción

El primer objetivo de este laboratorio es el agregar a la librería de ordenamiento `Ordenamiento.kt`, tres versiones del algoritmo Mergesort. El segundo objetivo es el realizar un programa cliente, llamado `PruebaOrdenamiento.kt`, que prueba los algoritmos de ordenamiento, y tiene la capacidad de graficar el tiempo de ejecución de los algoritmos versus el tamaño de la secuencia ordenada.

### 2. Actividades a realizar

En esta sección se presentan las actividades que debe realizar para cumplir los objetivos del laboratorio.

#### 2.1. Ampliación de la Librería de Ordenamiento

Se quiere ampliar la librería de algoritmos de ordenamiento con nuevos algoritmos que se describen a continuación.

##### 2.1.1. Algoritmos de Ordenamientos para Arreglos Pequeños

Debe agregar a la biblioteca tres procedimientos que ordenan arreglos con 2, 3 y 4 elementos. A continuación su descripción:

1. **ordenaDos**: procedimiento que recibe como entrada un arreglo de solo dos elementos y lo ordena con comparaciones de los elementos.
2. **ordenaTres**: procedimiento que recibe como entrada un arreglo de solo tres elementos y lo ordena con comparaciones de los elementos.
3. **ordenaCuatro**: procedimiento que recibe como entrada un arreglo de solo cuatro elementos y lo ordena con comparaciones de los elementos.

Si alguno de estos algoritmos recibe un arreglo mayor al tamaño indicado, imprime por la salida estándar un mensaje de error al usuario y finaliza la ejecución. Estos procedimientos reciben solo como entrada, un arreglo de elementos de tipo `Number`, como todos los algoritmos de ordenamiento de la librería.

### 2.1.2. Versiones del Mergesort

Se quiere que implemente tres versiones de Mergesort, siguiendo los pseudo códigos presentados en las referencias indicadas:

- **mb**: Versión de Mergesort contenida en [1], con la siguiente modificación:
  - Si el arreglo tiene tamaño uno, no hace nada.
  - Si el arreglo tiene tamaño dos, lo ordena con el procedimiento `ordenaDos`.
  - Si el arreglo tiene tamaño tres, lo ordena con el procedimiento `ordenaTres`.
  - Si el arreglo tiene tamaño cuatro, lo ordena con el procedimiento `ordenaCuatro`.
  - Si el arreglo tiene tamaño entre 5 y 100, incluyendo ambos, lo ordena con el procedimiento `insertionSort`.
- **ms**: Versión de Mergesort contenida en [2].
- **mi**: Mergesort-Iterativo que es presentado en la página 183 de [3].

En consecuencia, debe agregar a la librería `Ordenamiento.kt` tres procedimientos asociados a las tres versiones de Mergesort. Esos son: `mergesortInsertion`, `mergesort` y `mergesortIterativo`. Los tres procedimientos reciben solo como entrada, un arreglo de elementos tipo `Number`, como el resto de los algoritmos de ordenamiento de la librería. Para los procedimientos `mergesortInsertion` y `mergesort`, debe usar el mismo procedimiento `merge`.

Los códigos de referencias de los algoritmos de ordenamiento simples, del primer laboratorio, son los siguientes:

- **is**: Insertion sort.
- **bs**: Bubble sort.
- **ss**: Selection sort.

## 2.2. Descripción del Programa de Pruebas

En esta sección se describe el programa cliente que prueba los algoritmos de ordenamientos.

### 2.2.1. Tipos de Arreglos Generados

Las clases de arreglos que debe poder generar el programa cliente, junto con su identificador, son los siguientes:

| Identificador  | Tipo    | Descripción                                                                                                                                                          |
|----------------|---------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>random</b>  | Entero  | Arreglo de $N$ elementos de tipo entero, generados aleatoriamente en el intervalo $[0..N]$ .                                                                         |
| <b>randomd</b> | Double  | Arreglo de $N$ elementos de tipo double, generados aleatoriamente en el intervalo $[0.,1]$ .                                                                         |
| <b>sorted</b>  | Entero  | Arreglo de $N$ elementos de tipo entero, en donde los elementos están ordenados, tal que un arreglo tiene la forma $[1, 2, \dots, N]$ .                              |
| <b>sortedd</b> | Double  | Arreglo de $N$ elementos de tipo double, en donde los elementos en el intervalo $[0.,1]$ están ordenados.                                                            |
| <b>inv</b>     | Entero  | Arreglo de $N$ elementos de tipo entero, en donde los elementos están ordenados de forma inversa, tal que un arreglo tiene la forma $[N, \dots, 2, 1]$ .             |
| <b>zu</b>      | Binario | Arreglo de $N$ elementos cero y uno, generados aleatoriamente.                                                                                                       |
| <b>media</b>   | Entero  | Arreglo de $N$ elementos de tipo entero, en donde los elementos del arreglo tienen la forma $[1, 2, \dots, \lfloor N/2 \rfloor, \lfloor N/2 \rfloor, \dots, 2, 1]$ . |

### 2.2.2. Programa Cliente de Pruebas

El programa cliente se llama `PruebaOrdenamiento.kt`. Este programa se ejecuta por medio de un shell script, llamado `pord.sh`, que se ejecuta con la siguiente línea de comando:

```
> ./pord.sh [-t #num] [-a <alg>] [-s <tipo arr>] [-n <tamaños>] [-o <figura>]
```

La semántica de los parámetros de entrada es la siguiente:

- **-s:** Clase de arreglo a realizar, en donde el parámetro `tipo arr` es el identificador del arreglo. Si se indica una clase de arreglo que no existe, el programa muestra un mensaje de error y termina.
- **-a:** Algoritmos de ordenamiento a ejecutar. Si se escoge más de un algoritmo, los mismos deben estar separados por coma.
- **-t:** Número de intentos en el que se aplica el tipo de arreglo seleccionado.
- **-n:** Es una serie de números que indican los tamaños de los arreglos que se van a generar para su ordenamiento. Los números indicados deben ser diferentes, y deben estar en orden ascendente separadas por coma. En caso contrario, el programa debe abortar indicando un mensaje de error.
- **-o:** Indica que se debe mostrar un gráfico con el comportamiento del tiempo de ejecución de los algoritmos. Solo es válido si se indican dos o más tamaños de arreglo con la opción **-n**. En caso de haber indicado un solo arreglo, el programa aborta con un mensaje de error. El argumento `figura` es el nombre de la figura de formato `.png` que genera la librería de graficación, el cual no debe tener la extensión `.png`.

Todos los parámetros son obligatorios, excepto **-o**. En este caso obligatorios quiere decir, que si alguno de estos parámetros no se encuentra en la línea de comandos, entonces la ejecución se aborta con un mensaje de error. Este programa cliente trabaja de la siguiente manera:

1. Con el parámetro **-n** el usuario escoge varios tamaños de arreglos a ordenar.
2. Para un tamaño  $n$  de arreglo, se genera la clase de arreglo escogido de tamaño  $n$  de elementos a ordenar.
3. Este mismo arreglo se aplica a todos los algoritmos de ordenamiento, que hayan sido seleccionados por el usuario.
4. Cuando termina la ejecución de un algoritmo de ordenamiento, se toma el tiempo de ejecución del algoritmo.
5. El tiempo de ejecución de cada algoritmo, junto con el nombre del algoritmo, es mostrado por la salida estándar en el momento de finalizar el algoritmo.
6. Una vez terminado un algoritmo de ordenamiento, se debe verificar si los elementos del arreglo están ordenados correctamente.
7. En caso de que el algoritmo de ordenamiento no haya ordenado correctamente, se debe indicar un mensaje de error por la salida estándar y se aborta la ejecución del programa.
8. Los pasos anteriores, en donde se ordena un arreglo con un algoritmo de ordenamiento, lo llamamos *intento*. Se quiere que realice para todos los algoritmos de ordenamientos seleccionados, un número de intentos  $t$  indicado por el usuario. Observe que cada intento se realiza con el mismo arreglo.
9. Si  $t > 1$  then se debe mostrar, por la salida estándar, el tiempo promedio junto con la desviación estándar, de cada uno de los algoritmos de ordenamiento seleccionados.
10. Luego estos pasos se repiten para cada uno de los tamaños de arreglo.
11. Finalmente, si el usuario escogió mostrar la gráfica de tiempo de ejecución, entonces la misma se debe generar.

A continuación se presentan llamadas válidas del programa cliente:

```
> ./p0rd.sh -t 2 -s random -a is, ms -n 500,1000
```

Este comando ejecuta el programa cliente primero sobre un arreglo de 500, y luego sobre uno de 1000. Los arreglos están compuestos de números enteros generados al azar, teniendo que los algoritmos Insertion Sort y Mergesort se ejecutan dos veces sobre cada uno de los arreglos.

```
> ./p0rd.sh -s inv -n 20000,40000,50000 -t 3 -a ms, mi -o salidaInv
```

Al ejecutar este comando, el programa cliente aplica los algoritmos de ordenamiento Mergesort y Mergesort-Iterativo a tres arreglos de números enteros de tamaño 20000, 40000 y 50000, tres veces cada una. Luego genera un gráfico con los resultados de la corrida y el mismo queda guardado en el archivo **salidaInv.png**.

La opción **-o** debe generar una gráfica donde el eje  $y$  es el tiempo de ejecución de los algoritmos de ordenamiento en segundos y el eje  $x$  corresponde al tamaño de los arreglos indicados con la opción **-n**. La gráfica debe mostrar el comportamiento de todos los algoritmos de ordenamientos ejecutados. Para generar la gráfica se debe hacer uso de la librería de visualización **libPlotRuntime**, que fue publicada en clase. La Figura 1 muestra un ejemplo de cómo puede ser graficado el comportamiento de varios algoritmos de ordenamiento.

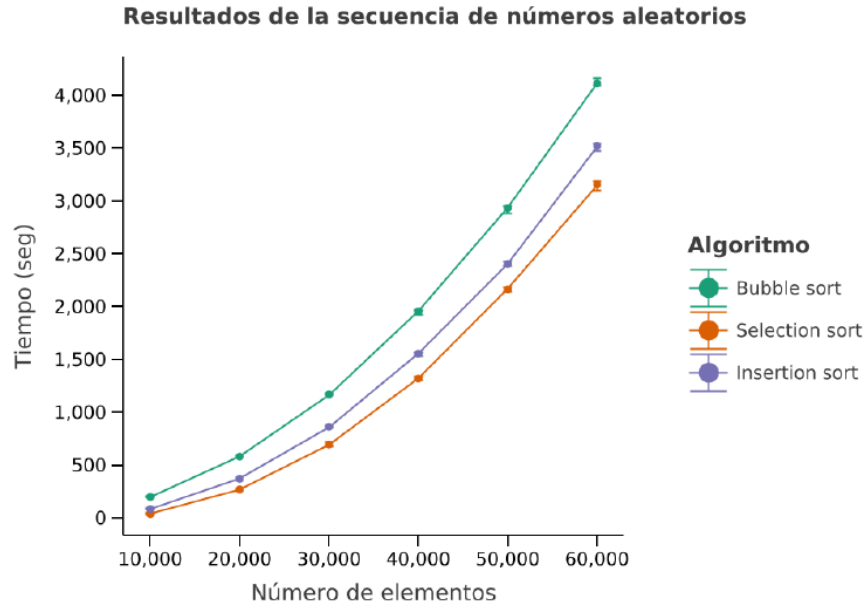


Figura 1: Ejemplo de una gráfica que muestra el comportamiento del tiempo de ejecución de tres algoritmos de ordenamiento, sobre seis arreglos de diferentes tamaños.

Todo el código debe usar la guía de estilo Kotlin indicada en clase. Asimismo, el código debe estar debidamente documentado. Debe crear un `Makefile` que compile todo el código. Para el desarrollo de este laboratorio solo está permitido el uso de la estructura de datos arreglos. No se pueden usar listas u otra estructura de datos de la librería de Kotlin, que no sea arreglo.

### 3. Condiciones de entrega

La entrega debe contener los archivos `Ordenamiento.kt`, `PruebaOrdenamiento.kt`, `Makefile` y `pord.sh`, y la carpeta `libPlotRuntime`. La versión final del código del laboratorio debe estar contenida en un archivo comprimido, con formato `tar.xz`, llamado `LabSem3_X_Y.tar.xz`, donde X y Y son el número de carné de los estudiantes. La entrega debe hacerse por la plataforma Classroom, antes de las 11:59pm del día lunes 25 de mayo de 2026.

### Referencias

- [1] BRASSARD, G., AND BRATLEY, P. *Fundamentals of Algorithmics*. Prentice Hall, 1996.
- [2] CORMEN, T. H., LEISERSON, C. E., RIVEST, R. L., AND STEIN, C. *Introduction to algorithms*. MIT press, 2022.
- [3] KALDEWALJ, A. *Programming: the derivation of algorithms*. Prentice-Hall, Inc., 1990.

---

Leonardo López / leonardolopez@usb.ve  
Kevin Oporto / kevinoportoh@gmail.com  
Mayo 2026